



Project Demo Planning

Date	29 June 2026
Team ID	N/A
Project Name	OptiCrop Smart Agricultural Production Optimization Engine
Maximum Marks	1 Mark

Project Demo Planning:

A well-structured demo plan ensures that the team presents the project effectively, covering all key aspects in a clear and organized manner. This document outlines the plan for demonstrating the project, including the flow of the demo, key features to highlight, and responsibilities of each team member.

S.No	Demo Section	Description	Duration (mins)	Responsible Member
1	Introduction	Personal introduction by the developer. Brief overview of the OptiCrop project — what it does, who it helps (farmers, researchers, policymakers), and why it was built.	1 min	Neelima
2	Problem Statement	Explain the agricultural challenges OptiCrop addresses — farmers lack a fast, data-driven way to determine which crop best suits their soil's actual nutrient levels and local climate, leading to poor yield, wasted resources, and avoidable financial loss.	1 min	Neelima
3	Solution Overview	Describe how OptiCrop solves the problem — a trained Random Forest classifier (99.55% accuracy) maps seven soil and climate inputs (N, P, K, temperature, humidity, pH, rainfall) to the most suitable crop, deployed as a Flask web application.	1 min	Neelima
4	Code Walkthrough	Walk through the key project files: dataset (Crop_recommendation.csv), Jupyter notebook (EDA, preprocessing, model comparison table, confusion matrix, pickle save), and app.py (Flask routes and prediction logic).	2 mins	Neelima
5	Live Application Demo	Open the Flask application at http://127.0.0.1:5000 . Navigate through all three pages — Home, About, Find Your Crop. Enter a rice profile (N=90, P=42, K=43, temperature=20.88, humidity=82.0, pH=6.5, rainfall=202.9) and demonstrate the live crop recommendation.	2 mins	Neelima
6	Results & Conclusion	Show the model comparison table (Random Forest 99.55% vs other algorithms). Summarize the project's contribution — complete end-to-end ML pipeline from raw data to deployed web application. Mention GitHub repository link and future scope (real-time weather API, cloud deployment).	1 min	Neelima

Demo Flow Summary:

Step	Activity	Notes
1	Introduction & Problem Statement	Introduce yourself (name, college, department). Explain the problem: farmers rely on guesswork for crop selection — leading to financial loss and wasted resources. State that OptiCrop solves this with a data-driven ML recommendation.
2	Solution Overview & Tech Stack	Briefly describe the solution: 7 soil/climate inputs → StandardScaler → Random Forest → crop recommendation. Mention the tech stack: Python, Scikit-learn, Flask, Matplotlib, Seaborn, Pandas, NumPy. Dataset: Crop Recommendation (Kaggle, 2,200 samples, 22 crops).
3	Code Walkthrough	Switch to VS Code. Show: (1) Crop_recommendation.csv — dataset structure. (2) EDA_and_Model_Training.ipynb — model comparison table, confusion matrix, pickle save. (3) app.py — Flask routes and prediction logic. Keep this section brief and focused.
4	Live Feature Demonstration	Run the Flask application (python app.py). Open http://127.0.0.1:5000 in the browser. Navigate Home → About → Find Your Crop. Enter the rice profile values and click Predict. Show the result: 'Recommended crop: Rice'. Application uses Flask, not Streamlit.
5	Q&A Session	Be ready to answer questions on: why Random Forest was chosen over other algorithms, what the 7 input features represent, how the model is loaded in Flask, how the project handles invalid inputs, and what future enhancements are planned.